FRENCH UNIVERSITY IN ARMENIA
Faculty of Applied Mathematics and Informatics

SMART AGRICULTURE FARM MANAGEMENT SYSTEM

Project Report and Documentation

Subject Databases 1

Students Mariam Ghevondyan,
Maria Danielyan

Professor Varazdat Avetisyan

April 6, 2026

Table of Contents

1. CONCEPTUAL DESIGN 4
1.1 Problem description 4
1.2 Requirements Analysis 5
Functional Requirements 5
Non-Functional Requirements 5
Data Requirements 5
1.3 Main data objects and Business rules 6
1.4 System Data Requirements 7
1.5 Flow-Charts 8
Activity Diagrams 8
Use-Case Diagram 11
Sequence Diagrams 11
2. LOGICAL DESIGN 15
2.1 Entity–Relationship (ER) diagram 15
2.2 Attributes, and relationships 16
2.3 Relationship cardinalities and constraints 19
2.4 Transformation of the ER model into a relational schema 21
2.5 Normalization Process (1NF, 2NF, 3NF) 23
2.6 Relational Algebra Operation 26
3. PHYSCIAL IMPLEMENTATION 28
3.1 SQL scripts 28
DDL part 28
DML part 31
DQL part 36
DCL part 40
3.2 Documentations 42
Views 42
Indexes 44
Triggers 46

Stored Procedures 47
3.3 Database deployment description 49
Project conclusions and results 51

1. CONCEPTUAL DESIGN

1.1 Problem description

We live in an age where modern agriculture increasingly relies on IoT technologies to monitor environmental conditions such as soil moisture, temperature, and humidity. However, managing large amounts of data from farms, fields, crops, sensors, irrigation systems, and workers can be complex without a centralized system.

Farmers often lack an efficient way to store and manage this data, leading to poor resource use and lower productivity. Therefore, a database system is needed to integrate IoT data and support better decision-making in farm management.

1.2 Requirements Analysis

The system must support the management of farm operations and integrate data collected from IoT sensors.

Functional Requirements

- Store and manage data about farms, fields, and crops
- Record environmental data from IoT sensors (soil moisture, temperature, humidity)
- Monitor field conditions in real time
- Manage irrigation systems and their status
- Assign and manage workers for different fields
- Record and track harvest data
- Support queries for analysis and decision-making

Non-Functional Requirements

- Ensure data accuracy and consistency
- Provide efficient data retrieval and performance
- Support scalability for large amounts of sensor data
- Ensure system reliability and availability

Data Requirements

- Farm information (name, location)
- Field details (size, assigned farm)
- Crop data (type, growth details)
- Sensor data (soil moisture, temperature, humidity)
- Irrigation system status
- Worker information
- Harvest records

1.3 Main data objects and Business rules

Data Object (Entity)	Description / Attributes	Business Rules / Relationships
Farm	FarmID, FarmName, Location	Each farm can have multiple fields.
Field	FieldID, FarmID, Area	Each field belongs to one farm.
Crop	CropID, CropName, GrowthDuration	A crop can be planted in multiple fields.
FieldCrop	FieldID, CropID, PlantingDate	Links crops to fields; a field can have multiple crops over time. Composite PK ensures uniqueness.
Worker	WorkerID, Name	Workers can work in multiple fields.
FieldWorker	FieldID, WorkerID	Assigns workers to fields; many-to-many relationship. Composite PK ensures uniqueness.
Sensor	SensorID, FieldID, SensorType	Each sensor belongs to one field; sensor type must be consistent.
SensorData	DataID, SensorID, Value, RecordedTime	Each reading belongs to a sensor; timestamp is mandatory.
IrrigationSystem	IrrigationID, FieldID, Status, LastUpdated	Each field has at most one irrigation system; status must be ‘ON’ or ‘OFF’.
Harvest	HarvestID, FieldID, CropID, Quantity, HarvestDate	Records quantity of crop harvested from a specific field on a specific date.

Table 1

1.4 System Data Requirements

Data Type	Description	Source / Input
Farm Data	Farm details: ID, name, location	Admin / System input
Field Data	Field ID, farm ID, area, crop type	Admin / Worker input
Crop Data	Crop ID, name, growth duration	System catalog / Admin input
Worker Data	Worker ID, name, role	Admin input
Field-Crop Assignments	Field ID, crop ID, planting date	Worker input / Admin
Sensor Data	Sensor ID, type, readings (value, timestamp)	Sensors (automatic)
Irrigation Data	Field ID, system status, last updated	Sensors / Manual input
Harvest Data	Field ID, crop ID, quantity, harvest date	Worker input / Admin

Table 2

1.5 Flow-Charts

Activity Diagrams

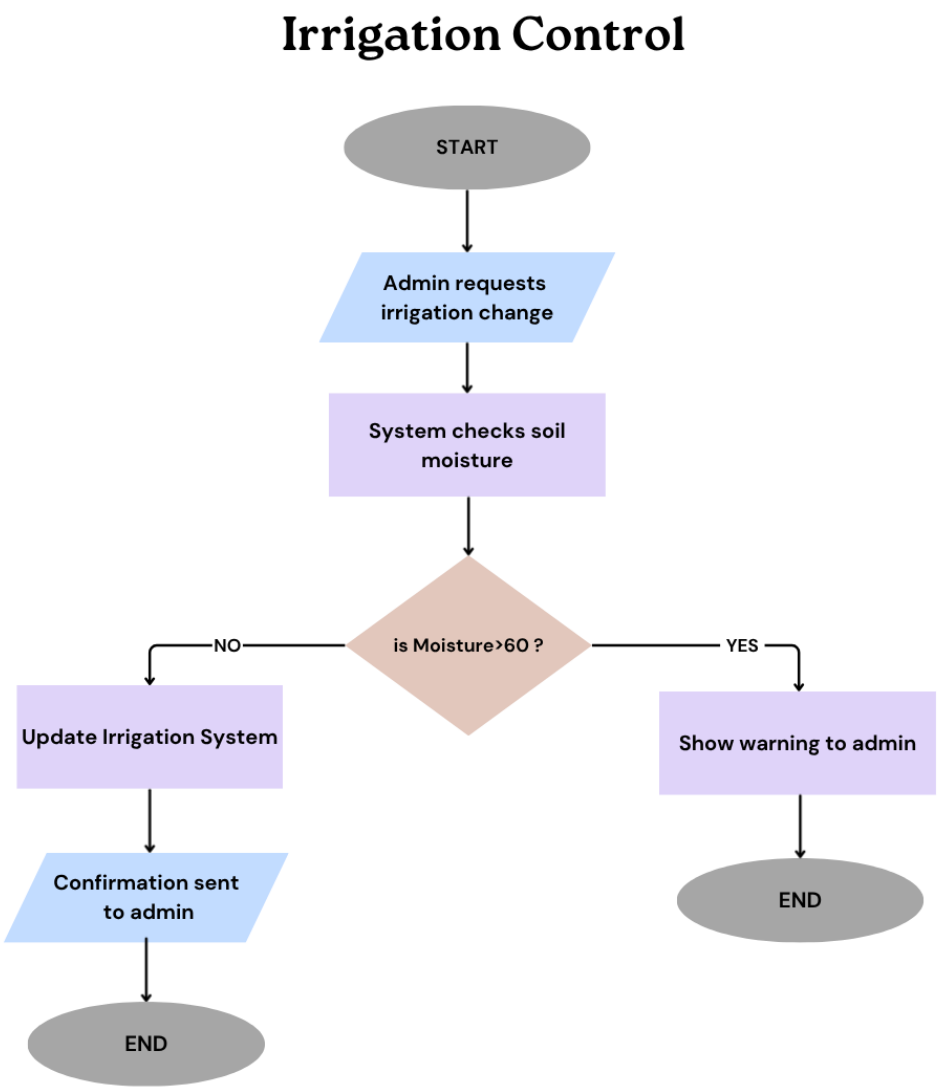

Figure 1

This flowchart illustrates the process of controlling irrigation based on soil moisture levels. The system checks sensor data and makes a decision whether irrigation should be activated or not. If conditions are suitable, the system updates the irrigation status; otherwise, it prevents activation and notifies the user.

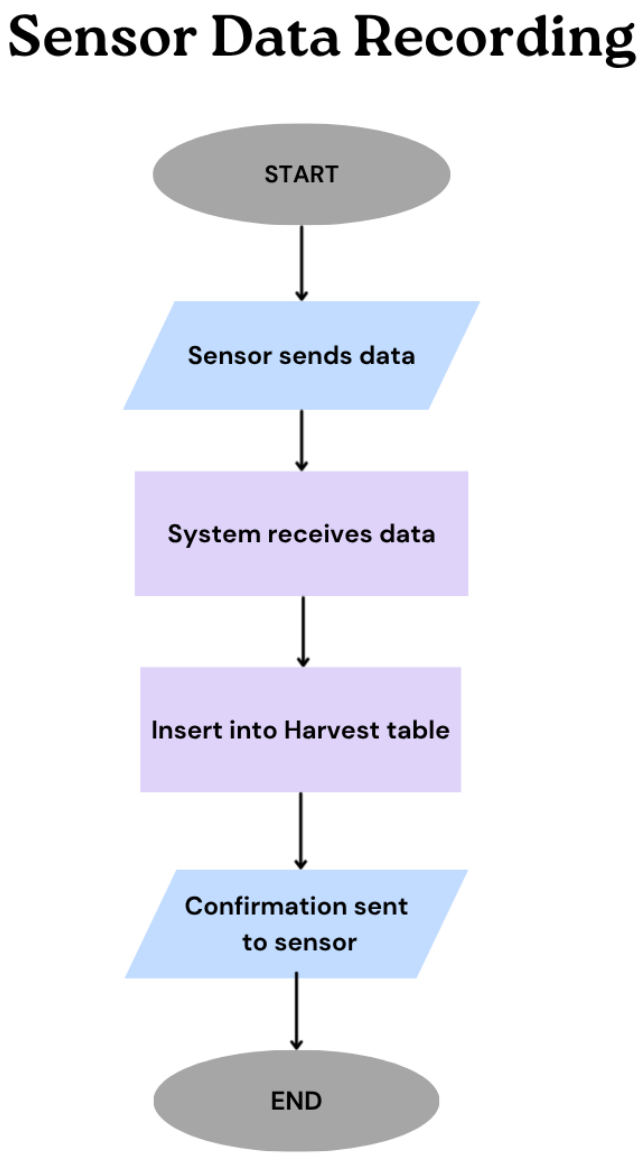

Figure 2

This flowchart shows how sensor data is collected and stored in the system. The sensor sends readings, which are processed and inserted into the database. The system then confirms that the data has been successfully recorded.

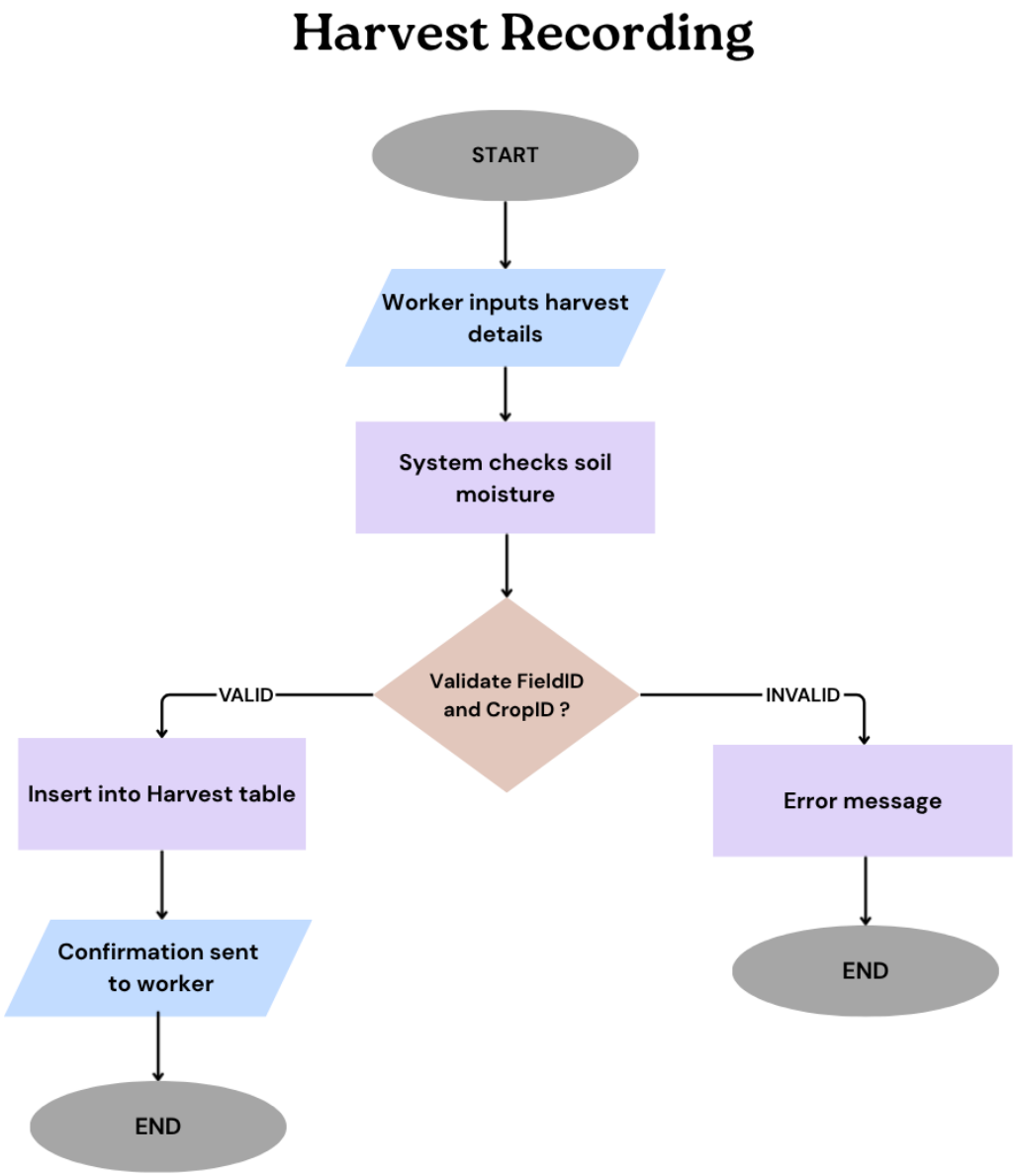

Figure 3

This flowchart represents the process of recording harvested crops by a worker. The system validates the entered data before storing it in the database. If the data is valid, the harvest is recorded; otherwise, an error message is returned.

Use-Case Diagram

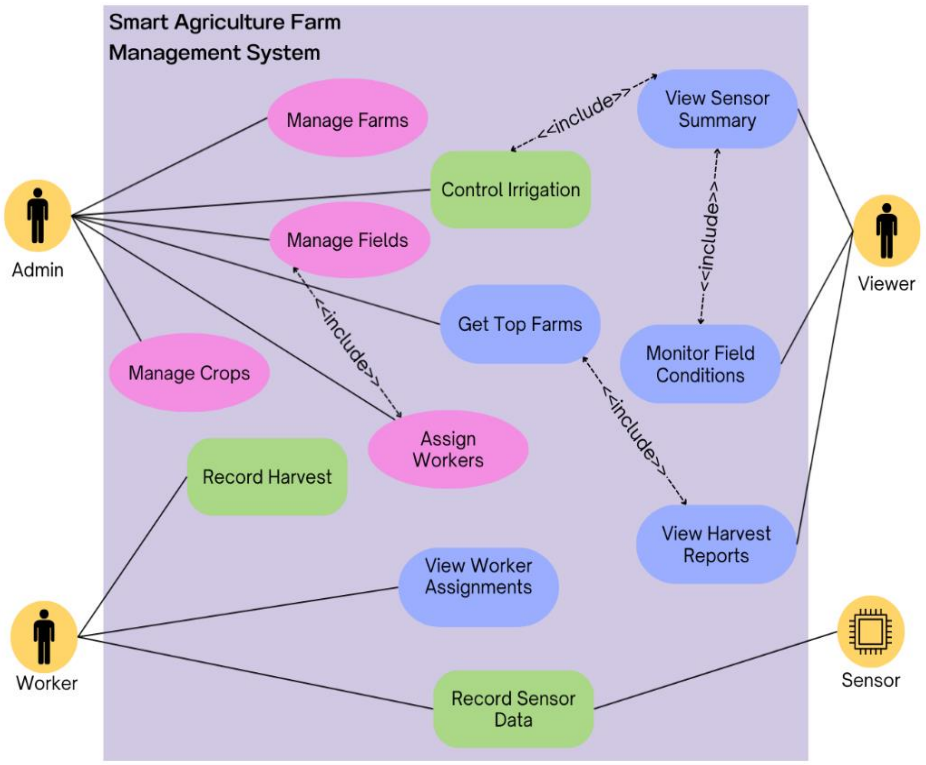

Figure 4

Use-Case Diagram explanation

The use-case diagram represents the interactions between different actors and the Smart Agriculture System. The main actors include Admin, Worker, Viewer, and Sensor, each performing specific tasks such as managing farms, recording data, and viewing reports. The diagram shows how core functionalities like irrigation control, sensor data recording, and harvest management are handled within the system. Additionally, relationships such as <<include>> highlight dependencies between use-cases, improving the clarity of system processes.

Sequence Diagrams

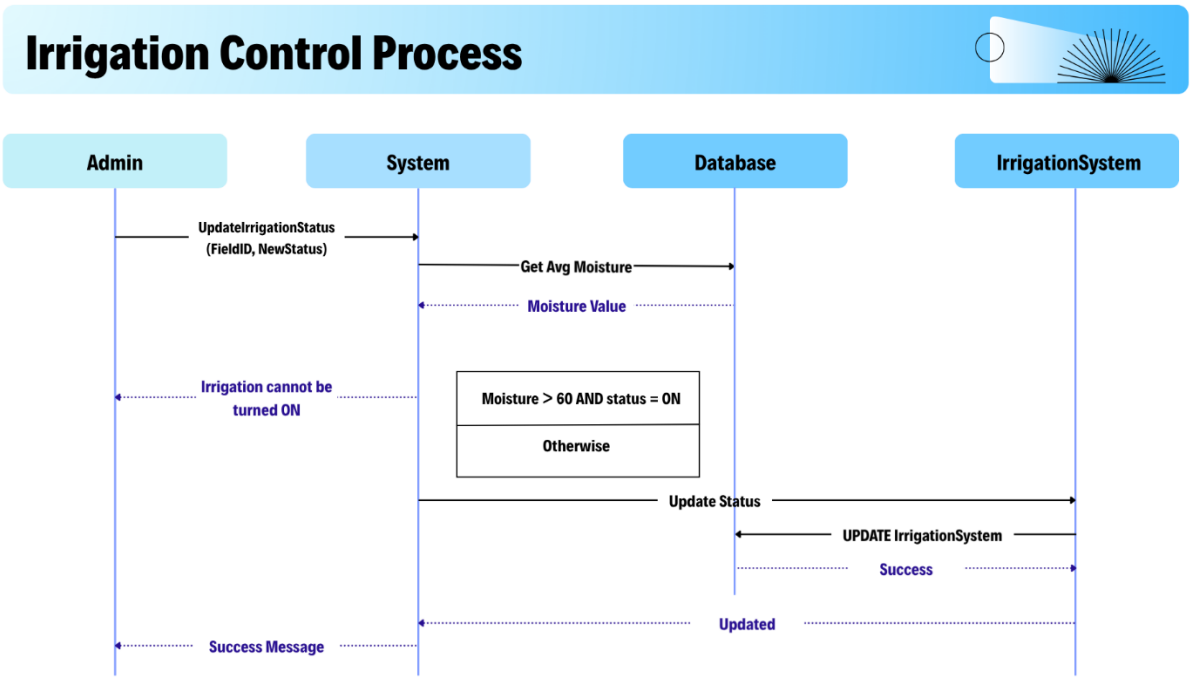

Figure 5

<u>Irrigation Control Process sequence diagram explanation</u>

This sequence diagram shows how an admin controls the irrigation system for a field. The system first retrieves soil moisture data from the database and evaluates it before performing any action. If the moisture level is too high, irrigation is not activated and a warning is returned. Otherwise, the system updates the irrigation status and confirms the successful operation.

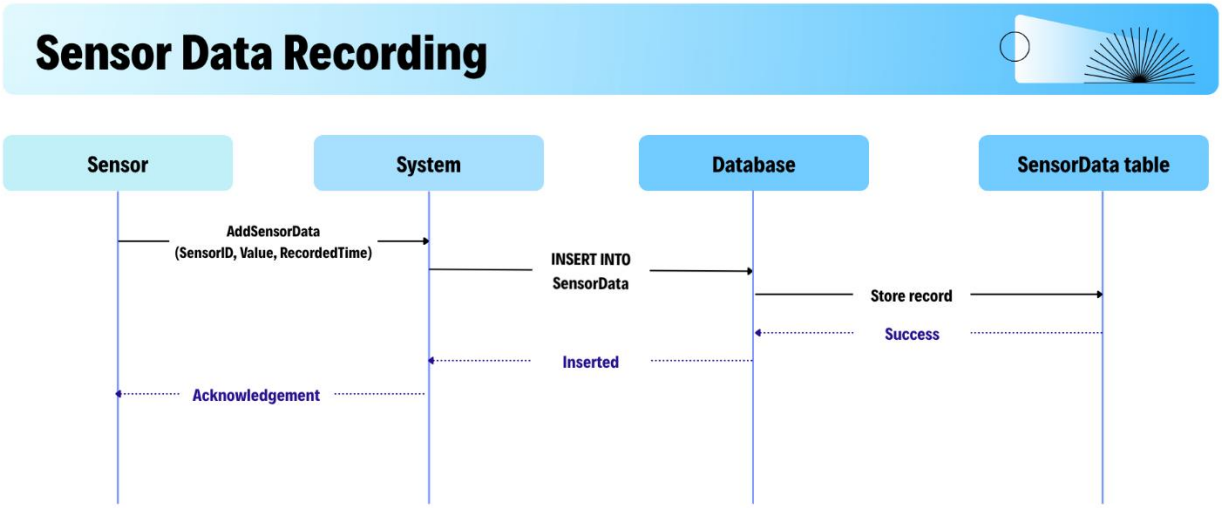

Figure 6

<u>Sensor Data Recording sequence diagram explanation</u>

This diagram represents how sensor devices automatically send environmental data to the system. The system receives the sensor readings and stores them in the database using the appropriate procedure. After successful insertion, a confirmation is returned. This process ensures continuous and real-time monitoring of field conditions.

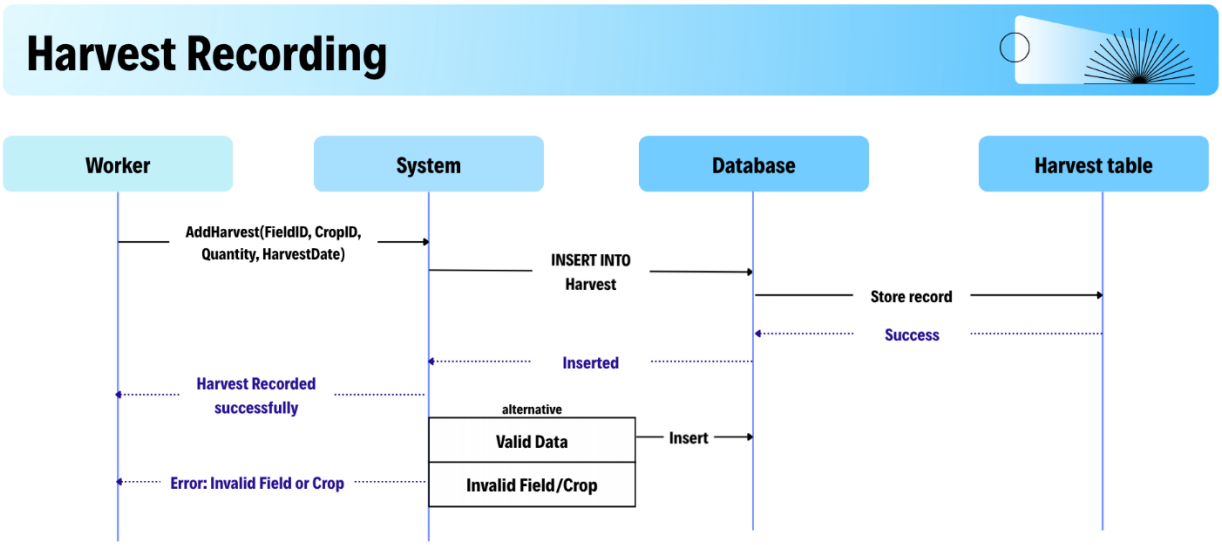

Figure 7

<u>Harvest Recording Process sequence diagram explanation</u>

This sequence diagram illustrates how a worker records harvested crops in the system. The worker submits harvest details, which the system processes and inserts into the database. The system may validate the input before storing it to ensure data integrity. Once completed, a confirmation message is sent back to the worker.

2. LOGICAL DESIGN

The figure above presents the Entity–Relationship (ER) diagram of the Smart Agriculture database system. It visually represents the main entities, their attributes, and the relationships between them.

2.1 Entity–Relationship (ER) diagram

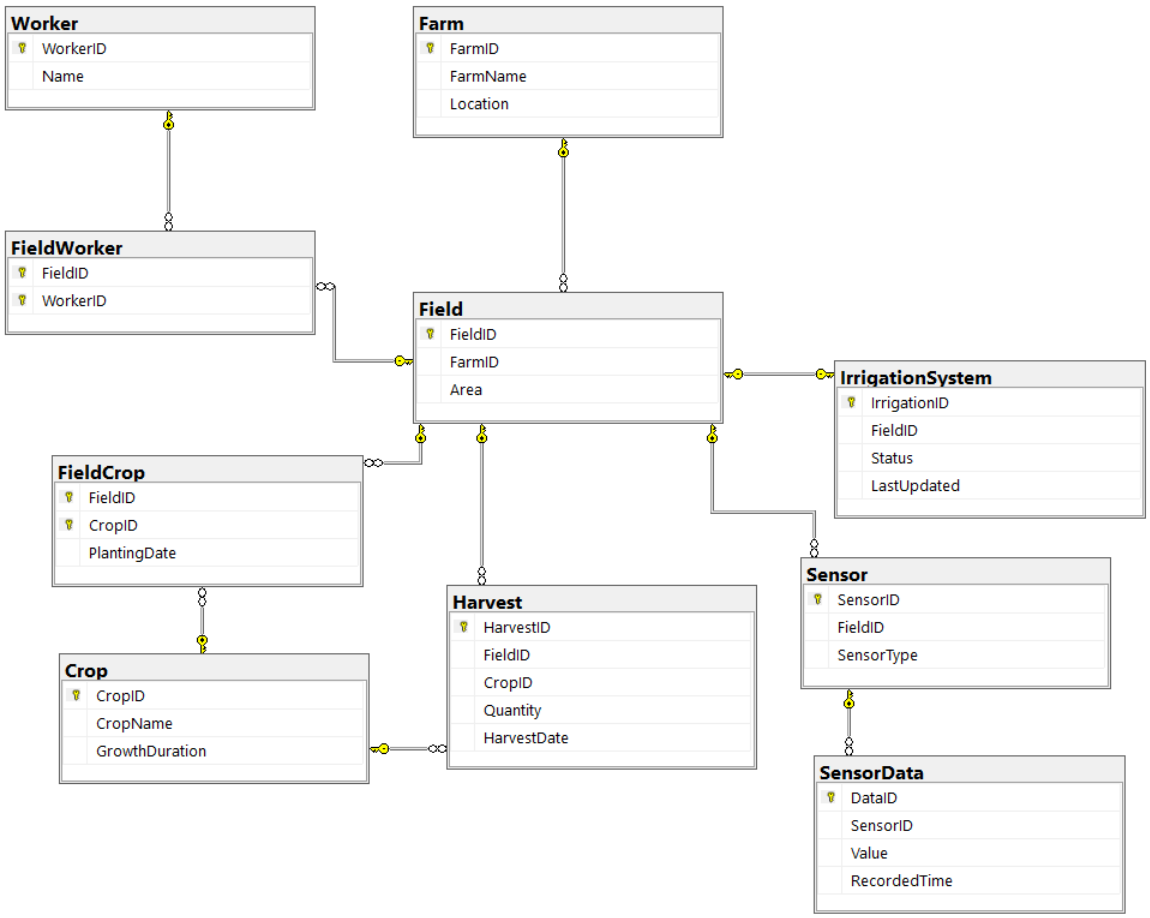

Figure 8

2.2 Attributes, and relationships

Table: Farm

Column	Column	Column	Column
FarmID	INT	PK	Unique farm identifier
FarmID	VARCHAR(100)	NOT NULL	Name of the farm
FarmID	VARCHAR(100)	NOT NULL	Farm location

Table 3

Table: Field

Column	Data Type	Constraints	Description
FieldID	INT	PK	Unique field identifier
FarmID	INT	FK, NOT NULL	Reference to Farm
Area	DECIMAL(10,2)	NOT NULL	Field area in hectares

Table 4

Table: FieldCrop

Column	Column	Column	Column
FieldID	INT	PK, FK	Reference to Field
CropID	INT	PK, FK	Reference to Crop
PlantingDate	DATE	NOT NULL	Date of planting

Table 5

Table: FieldWorker

Column	Data Type	Constraints	Description
FieldID	INT	PK, FK	Reference to Field
WorkerID	INT	PK, FK	Reference to Worker

Table 6

Table: Sensor

Column	Data Type	Constraints	Description
SensorID	INT	PK	Unique sensor identifier
FieldID	INT	FK, NOT NULL	Reference to Field
SensorType	VARCHAR(50)	NOT NULL	Type of sensor

Table 7

Table: SensorData

Column	Data Type	Constraints	Description
DataID	INT	PK	Unique data record ID
SensorID	INT	FK, NOT NULL	Reference to Sensor
Value	DECIMAL(6,2)	NOT NULL	Sensor reading value
RecordedTime	DATETIME	NOT NULL	Time of measurement

Table 8

Table: IrrigationSystem

Column	Data Type	Constraints	Description
IrrigationID	INT	PK	Unique irrigation ID

Table: IrrigationSystem

Column	Data Type	Constraints	Description
FieldID	INT	FK, UNIQUE, NOT NULL	One system per field
Status	VARCHAR(10)	CHECK ('ON','OFF')	Irrigation status
LastUpdated	DATETIME	DEFAULT GETDATE()	Last update timestamp

Table 9

Table: Harvest

Column	Data Type	Constraints	Description
HarvestID	INT	PK	Unique harvest record ID
FieldID	INT	FK, NOT NULL	Reference to Field
CropID	INT	FK, NOT NULL	Reference to Crop
Quantity	DECIMAL(10,2)	NOT NULL	Harvest quantity
HarvestDate	DATE	NOT NULL	Date of harvest

Table 10

2.3 Relationship cardinalities and constraints

The system enforces referential integrity using foreign keys and constraints. One-to-many relationships are implemented through foreign keys, while many-to-many relationships are resolved using associative tables such as FieldCrop and FieldWorker. One-to-one relationships are enforced using unique constraints on foreign keys, such as in the IrrigationSystem table.

Relationship	Relationship	Relationship	Relationship	Relationship
Farm → Field	1 to *	1:N	ON DELETE CASCADE	One farm can have many fields; deleting a farm removes its fields
Field → Sensor	1 to *	1:N	ON DELETE CASCADE	A field can contain multiple sensors
Sensor → SensorData	1 to *	1:N	ON DELETE CASCADE	A sensor generates multiple readings over time
Field → IrrigationSystem	1 to 1	1:1 (UNIQUE FK)	ON DELETE CASCADE	Each field has exactly one irrigation system
Field → Harvest	1 to *	1:N	No cascade	A field can have multiple harvest records
Crop → Harvest	1 to *	1:N	No cascade	A crop can appear in multiple harvests
Field → FieldCrop	1 to *	1:N	ON DELETE CASCADE	A field can have multiple crop assignments

Crop → FieldCrop	1 to *	1:N	ON DELETE CASCADE	A crop can be planted in multiple fields
Field → FieldWorker	1 to *	1:N	ON DELETE CASCADE	A field can have multiple workers
Worker → FieldWorker	1 to *	1:N	ON DELETE CASCADE	A worker can work on multiple fields
Field → Sensor (Moisture logic)	0..1 to *	1:N	No cascade	Some sensors (moisture) trigger irrigation logic
Sensor → SensorAlerts	0..1 to *	1:N	ON DELETE CASCADE	Sensors may generate alerts when abnormal values occur

Table 11

2.4 Transformation of the ER model into a relational schema

The ER model is transformed into a relational schema using standard transformation rules:

- Each entity is converted into a table with attributes as columns.
- Primary keys uniquely identify each record.
- Foreign keys represent relationships between entities.
- Many-to-many relationships are resolved using associative tables (FieldCrop, FieldWorker).

```
SCHEMA — Relational Schema

1. Farm (
   FarmID PK, FarmName, Location
)

2. Field (
   FieldID PK, FarmID FK->Farm CASCADE, Area
)

3. Crop (
   CropID PK, CropName, GrowthDuration
)

4. Worker (
   WorkerID PK, Name
)

5. FieldCrop (
   FieldID PK, FK->Field CASCADE, CropID PK, FK->Crop CASCADE, PlantingDate
)

6. FieldWorker (
   FieldID PK, FK->Field CASCADE, WorkerID PK, FK->Worker CASCADE
)

7. Sensor (
   SensorID PK, FieldID FK->Field CASCADE, SensorType
)

8. SensorData (
   DataID PK, SensorID FK->Sensor CASCADE, Value, RecordedTime
)

9. IrrigationSystem (
   IrrigationID PK, FieldID FK->Field UNIQUE CASCADE, Status CHK, LastUpdated
)

10. Harvest (
   HarvestID PK, FieldID FK->Field, CropID FK->Crop, Quantity CHK, HarvestDate
)

11. SensorAlerts (AlertID PK, SensorID FK->Sensor CASCADE, Value, RecordedTime, AlertType
```

- One-to-one relationships are implemented using UNIQUE constraints (IrrigationSystem).
- Data integrity is enforced through constraints such as NOT NULL, CHECK, and UNIQUE.

2.5 Normalization Process (1NF, 2NF, 3NF)

I. First Normal Form — 1NF

Definition:

Every column contains atomic (indivisible) values; each row is uniquely identified by a primary key.

Applied in the system:

- All attributes store atomic values (no arrays, lists, or repeating groups).
- Each table has a primary key (e.g., FarmID, FieldID, SensorID).
- No multi-valued attributes exist.

Examples:

- ***SensorData.Value stores a single numeric value per record.***
- ***FieldCrop stores one crop per row (no multiple crops in one field row).***

II. Second Normal Form — 2NF

Definition:

In 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependency).

Applied in the system:

- Most tables use single-column primary keys → partial dependency is not possible.
- In associative tables:
    - FieldCrop(FieldID, CropID → PlantingDate)
    - FieldWorker(FieldID, WorkerID)

All non-key attributes depend on the full composite key.

Examples:

- ***PlantingDate depends on both FieldID and CropID.***
- ***SensorType depends only on SensorID.***

III. Third Normal Form — 3NF

Definition:

In 2NF and no non-key attribute is transitively dependent on the primary key.

Potential Transitive Dependency	Problem if Not Eliminated	Resolution Applied
FarmName stored in Field	Changing farm name requires updating multiple rows	Field stores only FarmID; FarmName retrieved via JOIN
CropName stored in Harvest	Redundant storage and inconsistency	Harvest stores CropID only
Worker name in FieldWorker	Duplication across assignments	Worker data stored separately
Sensor type in SensorData	Repetition and inconsistency	SensorData references SensorID only
Field area in SensorData	Redundant data	Field info retrieved via JOIN

Table 12

! The database design follows normalization principles up to Third Normal Form (3NF), ensuring minimal redundancy, improved data consistency, and efficient update operations.

2.6 Relational Algebra Operation

Selection σ — filter rows (Q1–Q3)

Q#	Op	Formal Expression	SQL Clause	Result
Q1	σ	σ_{Area > 50}(Field)	WHERE Area > 50	Large fields
Q2	σ	σ_{GrowthDuration > 120}(Crop)	WHERE GrowthDuration > 120	Long-duration crops
Q3	σ	σ_{Status = 'ON'}(IrrigationSystem)	WHERE Status = 'ON'	Active irrigation systems

Table 13

Projection π — return columns (Q4–Q6)

Q#	Op	Formal Expression	SQL Clause	Result
Q4	π	π_{FarmName}(Farm)	SELECT FarmName	Farm names
Q5	π	π_{Name}(Worker)	SELECT Name	Worker names
Q6	π	π_{SensorType}(Sensor)	SELECT SensorType	Sensor types

Table 14

Join ⋈ — combine tables (Q7–Q9)

Q#	Op	Formal Expression	Result
Q7	⋈	Farm ⋈ Field	Farms with their fields
Q8	⋈	FieldCrop ⋈ Crop	Crops planted in fields
Q9	⋈	SensorData ⋈ Sensor	Sensor readings with type

*Table **15***

Aggregation γ — grouping (Q10–Q12)

Q#	Op	Formal Expression	Result
Q10	γ	γ_{FarmName; SUM(Quantity)}(Harvest ⋈ Field ⋈ Farm)	Total harvest per farm
Q11	γ	γ_{FieldID; AVG(Value)}(SensorData ⋈ Sensor)	Average sensor value per field
Q12	γ	γ_{FieldID; COUNT(WorkerID)}(FieldWorker)	Workers per field

Table 16

Union ∪ (Q13)

Q#	Op	Formal Expression	Result
Q13	∪	π_{FarmID}(Farm) ∪ π_{WorkerID}(Worker)	All IDs from farms and workers

Table 17

Intersection ∩ (Q14)

Q#	Op	Formal Expression	Result
Q14	∩	π_{FieldID}(Field) ∩ π_{FieldID}(IrrigationSystem)	Fields that have irrigation

Table 18

Difference − (Q15)

Q#	Op	Formal Expression	Result
Q15	−	π_{FieldID}(Field) − π_{FieldID}(IrrigationSystem)	Fields without irrigation

Table 19

3. PHYSCIAL IMPLEMENTATION

3.1 SQL scripts

DDL part

Table Name	Attributes	Primary Key	Foreign Keys	Description
Farm	FarmID, FarmName, Location	FarmID	—	Stores information about farms
Crop	CropID, CropName, GrowthDuration	CropID	—	Stores crop types and their growth duration
Worker	WorkerID, Name	WorkerID	—	Stores worker details
Field	FieldID, FarmID, Area	FieldID	FarmID → Farm(FarmID)	Represents fields belonging to farms
FieldCrop	FieldID, CropID, PlantingDate	(FieldID, CropID)	FieldID → Field(FieldID), CropID → Crop(CropID)	Tracks which crops are planted in which fields
Sensor	SensorID, FieldID, SensorType	SensorID	FieldID → Field(FieldID)	Sensors installed in fields
SensorData	DataID, SensorID, Value, RecordedTime	DataID	SensorID → Sensor(SensorID)	Stores sensor readings over time
IrrigationSystem	IrrigationID, FieldID, Status, LastUpdated	IrrigationID	FieldID → Field(FieldID)	Controls irrigation system per field
FieldWorker	FieldID, WorkerID	(FieldID, WorkerID)	FieldID → Field(FieldID), WorkerID → Worker(WorkerID)	Assigns workers to fields
Harvest	HarvestID, FieldID, CropID, Quantity, HarvestDate	HarvestID	FieldID → Field(FieldID), CropID → Crop(CropID)	Records harvest data

Table 20

```sql
-- ================================================================
-- CREATING TABLES (DDL)
-- ================================================================


PRINT '================================================================';
PRINT 'DDL — CREATING TABLES';
PRINT '================================================================';
GO


USE SmartAgricultureDB;
GO

-- Drop tables if they exist (in reverse dependency order)
IF OBJECT_ID('Harvest', 'U') IS NOT NULL DROP TABLE Harvest;
IF OBJECT_ID('FieldWorker', 'U') IS NOT NULL DROP TABLE FieldWorker;
IF OBJECT_ID('IrrigationSystem', 'U') IS NOT NULL DROP TABLE IrrigationSystem;
IF OBJECT_ID('SensorData', 'U') IS NOT NULL DROP TABLE SensorData;
IF OBJECT_ID('Sensor', 'U') IS NOT NULL DROP TABLE Sensor;
IF OBJECT_ID('FieldCrop', 'U') IS NOT NULL DROP TABLE FieldCrop;
IF OBJECT_ID('Field', 'U') IS NOT NULL DROP TABLE Field;
IF OBJECT_ID('Worker', 'U') IS NOT NULL DROP TABLE Worker;
IF OBJECT_ID('Crop', 'U') IS NOT NULL DROP TABLE Crop;
IF OBJECT_ID('Farm', 'U') IS NOT NULL DROP TABLE Farm;
GO

CREATE TABLE Farm (
   FarmID INT PRIMARY KEY,
   FarmName VARCHAR(100),
   Location VARCHAR(100)
);
GO

CREATE TABLE Crop (
   CropID INT PRIMARY KEY,
   CropName VARCHAR(100),
   GrowthDuration INT
);
GO

CREATE TABLE Worker (
   WorkerID INT PRIMARY KEY,
   Name VARCHAR(100)
);
GO

CREATE TABLE Field (
   FieldID INT PRIMARY KEY,
   FarmID INT,
   Area DECIMAL(10,2),
   FOREIGN KEY (FarmID) REFERENCES Farm(FarmID)
);
GO

CREATE TABLE FieldCrop (
   FieldID INT,
```

```
    CropID INT,
    PlantingDate DATE,
    PRIMARY KEY (FieldID, CropID),
    FOREIGN KEY (FieldID) REFERENCES Field(FieldID),
    FOREIGN KEY (CropID) REFERENCES Crop(CropID)
);
GO

CREATE TABLE Sensor (
    SensorID INT PRIMARY KEY,
    FieldID INT,
    SensorType VARCHAR(50),
    FOREIGN KEY (FieldID) REFERENCES Field(FieldID)
);
GO

CREATE TABLE SensorData (
    DataID INT PRIMARY KEY,
    SensorID INT,
    Value DECIMAL(6,2),
    RecordedTime DATETIME,
    FOREIGN KEY (SensorID) REFERENCES Sensor(SensorID)
);
GO

CREATE TABLE IrrigationSystem (
    IrrigationID INT PRIMARY KEY,
    FieldID INT UNIQUE,
    Status VARCHAR(10) CHECK (Status IN ('ON','OFF')),
    LastUpdated DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (FieldID) REFERENCES Field(FieldID)
);
GO

CREATE TABLE FieldWorker (
    FieldID INT,
    WorkerID INT,
    PRIMARY KEY (FieldID, WorkerID),
    FOREIGN KEY (FieldID) REFERENCES Field(FieldID),
    FOREIGN KEY (WorkerID) REFERENCES Worker(WorkerID)
);
GO

CREATE TABLE Harvest (
    HarvestID INT PRIMARY KEY,
    FieldID INT,
    CropID INT,
    Quantity DECIMAL(10,2),
    HarvestDate DATE,
    FOREIGN KEY (FieldID) REFERENCES Field(FieldID),
    FOREIGN KEY (CropID) REFERENCES Crop(CropID)
);
GO
```

DML part

Table Name	Number of Records	Key Data Inserted	Description
Farm	43	FarmID, FarmName, Location	43 farms across Armenia with unique names
Crop	43	CropID, CropName, GrowthDuration	Variety of crops (grains, fruits, vegetables) with growth periods
Worker	43	WorkerID, Name	Workers assigned to farms/fields
Field	43	FieldID, FarmID, Area	Fields linked to farms with different sizes
FieldCrop	43	FieldID, CropID, PlantingDate	Each field assigned one crop with planting date
Sensor	43	SensorID, FieldID, SensorType	Sensors (Moisture, Temperature, Humidity) installed in fields
SensorData	43	DataID, SensorID, Value, RecordedTime	Sensor readings recorded over time
IrrigationSystem	43	IrrigationID, FieldID, Status	Irrigation system status (ON/OFF) per field
FieldWorker	43+	FieldID, WorkerID	Workers assigned to fields (some fields have multiple workers)
Harvest	43	HarvestID, FieldID, CropID, Quantity, HarvestDate	Harvest records with yield quantities and dates

Table 21

```
-- ================================================================
-- INSERTING DATA (DML)
-- ================================================================


PRINT '================================================================';
PRINT 'DML — INSERTING DATA';
PRINT '================================================================';
GO

INSERT INTO Farm (FarmID, FarmName, Location) VALUES
(1,'Green Farm','Armenia'),
(2,'Sunrise Farm','Armenia'),
(3,'Riverbend Farm','Armenia'),
(4,'Golden Field','Armenia'),
(5,'Valley Farm','Armenia'),
(6,'Highland Farm','Armenia'),
(7,'Blue Sky Farm','Armenia'),
```

```
(8,'Maple Farm','Armenia'),
(9,'Sunny Acres','Armenia'),
(10,'Evergreen Farm','Armenia'),
(11,'Silver Creek','Armenia'),
(12,'Red Oak Farm','Armenia'),
(13,'Golden Harvest','Armenia'),
(14,'Meadowlands','Armenia'),
(15,'Pine Hill Farm','Armenia'),
(16,'Autumn Farm','Armenia'),
(17,'Crystal Lake','Armenia'),
(18,'Cedar Grove','Armenia'),
(19,'Springfield Farm','Armenia'),
(20,'Hilltop Farm','Armenia'),
(21,'Willow Farm','Armenia'),
(22,'Cherry Blossom','Armenia'),
(23,'Sunset Farm','Armenia'),
(24,'River View','Armenia'),
(25,'Mountain Farm','Armenia'),
(26,'Oakwood Farm','Armenia'),
(27,'Blueberry Farm','Armenia'),
(28,'Golden Valley','Armenia'),
(29,'Maple Leaf Farm','Armenia'),
(30,'Silver Hill','Armenia'),
(31,'Everest Farm','Armenia'),
(32,'Pioneer Farm','Armenia'),
(33,'Harmony Farm','Armenia'),
(34,'Autumn Leaves','Armenia'),
(35,'Sunflower Farm','Armenia'),
(36,'Crystal Ridge','Armenia'),
(37,'Clover Field','Armenia'),
(38,'Oakridge Farm','Armenia'),
(39,'Riverbend South','Armenia'),
(40,'Maple Grove','Armenia'),
(41,'Green Valley','Armenia'),
(42,'Golden Meadow','Armenia'),
(43,'Blue Horizon','Armenia');
GO

INSERT INTO Crop (CropID, CropName, GrowthDuration) VALUES
(1,'Wheat',120),(2,'Corn',90),(3,'Potato',80),(4,'Rice',150),(5,'Barley',110),
(6,'Soybean',100),(7,'Tomato',75),(8,'Cucumber',60),(9,'Carrot',70),(10,'Onion',90),
(11,'Pea',60),(12,'Lettuce',50),(13,'Spinach',40),(14,'Pumpkin',100),(15,'Cabbage',85),
(16,'Strawberry',70),(17,'Blueberry',120),(18,'Apple',150),(19,'Pear',140),(20,'Cherry',130),
(21,'Plum',120),(22,'Grapes',110),(23,'Banana',200),(24,'Orange',180),(25,'Lemon',160),
(26,'Peach',140),(27,'Mango',150),(28,'Avocado',200),(29,'Broccoli',90),(30,'Cauliflower',85),
(31,'Radish',40),(32,'Beetroot',80),(33,'Sweet Potato',110),(34,'Cornflower',50),(35,'Melon',90),
(36,'Watermelon',100),(37,'Pumpkin Large',120),(38,'Sugarcane',200),(39,'Rye',100),(40,'Millet',85),
(41,'Quinoa',90),(42,'Oats',100),(43,'Sunflower',75);
GO

INSERT INTO Worker (WorkerID, Name) VALUES
(1,'Arman'),(2,'Anna'),(3,'David'),(4,'Karen'),(5,'Narek'),
(6,'Sofia'),(7,'Levon'),(8,'Tigran'),(9,'Mariam'),(10,'Hovhannes'),
(11,'Ani'),(12,'Vahan'),(13,'Lilit'),(14,'Garen'),(15,'Elena'),
(16,'Ruben'),(17,'Narine'),(18,'Vardan'),(19,'Susanna'),(20,'Artur'),
(21,'Emma'),(22,'Samvel'),(23,'Tamara'),(24,'Hayk'),(25,'Alina'),
(26,'Grigor'),(27,'Marine'),(28,'Arpi'),(29,'Lev'),(30,'Siranush'),
(31,'Ara'),(32,'Tatevik'),(33,'Karen S'),(34,'Hermine'),(35,'Raffi'),
(36,'Gayane'),(37,'Hrayr'),(38,'Elvina'),(39,'Sevak'),(40,'Armine'),
```

```
(41,'Shant'),(42,'Nana'),(43,'Vigen');
GO

INSERT INTO Field (FieldID, FarmID, Area) VALUES
(1,1,50.5),(2,1,30.0),(3,2,45.2),(4,2,60.0),(5,3,55.5),
(6,3,35.5),(7,4,40.0),(8,4,25.0),(9,5,70.0),(10,5,60.5),
(11,6,50.0),(12,6,45.0),(13,7,30.0),(14,7,40.0),(15,8,55.0),
(16,8,35.5),(17,9,60.0),(18,9,50.0),(19,10,45.0),(20,10,55.0),
(21,11,65.0),(22,11,50.5),(23,12,40.0),(24,12,30.0),(25,13,60.0),
(26,13,45.0),(27,14,35.0),(28,14,50.0),(29,15,55.0),(30,15,45.0),
(31,16,40.0),(32,16,35.5),(33,17,60.0),(34,17,50.0),(35,18,45.5),
(36,18,55.0),(37,19,40.0),(38,19,50.0),(39,20,60.0),(40,20,45.0),
(41,21,55.5),(42,21,50.0),(43,22,60.0);
GO

INSERT INTO FieldCrop (FieldID, CropID, PlantingDate) VALUES
(1,1,'2025-03-01'),(2,2,'2025-03-02'),(3,3,'2025-03-03'),(4,4,'2025-03-04'),(5,5,'2025-03-05'),
(6,6,'2025-03-06'),(7,7,'2025-03-07'),(8,8,'2025-03-08'),(9,9,'2025-03-09'),(10,10,'2025-03-10'),
(11,11,'2025-03-11'),(12,12,'2025-03-12'),(13,13,'2025-03-13'),(14,14,'2025-03-14'),(15,15,'2025-03-15'),
(16,16,'2025-03-16'),(17,17,'2025-03-17'),(18,18,'2025-03-18'),(19,19,'2025-03-19'),(20,20,'2025-03-20'),
(21,21,'2025-03-21'),(22,22,'2025-03-22'),(23,23,'2025-03-23'),(24,24,'2025-03-24'),(25,25,'2025-03-25'),
(26,26,'2025-03-26'),(27,27,'2025-03-27'),(28,28,'2025-03-28'),(29,29,'2025-03-29'),(30,30,'2025-03-30'),
(31,31,'2025-03-31'),(32,32,'2025-04-01'),(33,33,'2025-04-02'),(34,34,'2025-04-03'),(35,35,'2025-04-04'),
(36,36,'2025-04-05'),(37,37,'2025-04-06'),(38,38,'2025-04-07'),(39,39,'2025-04-08'),(40,40,'2025-04-09'),
(41,41,'2025-04-10'),(42,42,'2025-04-11'),(43,43,'2025-04-12');
GO

INSERT INTO Sensor (SensorID, FieldID, SensorType) VALUES
(1,1,'Moisture'),(2,1,'Temperature'),(3,2,'Humidity'),(4,2,'Moisture'),(5,3,'Temperature'),
(6,3,'Humidity'),(7,4,'Moisture'),(8,4,'Temperature'),(9,5,'Humidity'),(10,5,'Moisture'),
(11,6,'Temperature'),(12,6,'Humidity'),(13,7,'Moisture'),(14,7,'Temperature'),(15,8,'Humidity'),
(16,8,'Moisture'),(17,9,'Temperature'),(18,9,'Humidity'),(19,10,'Moisture'),(20,10,'Temperature'),
(21,11,'Humidity'),(22,11,'Moisture'),(23,12,'Temperature'),(24,12,'Humidity'),(25,13,'Moisture'),
(26,13,'Temperature'),(27,14,'Humidity'),(28,14,'Moisture'),(29,15,'Temperature'),(30,15,'Humidity'),
(31,16,'Moisture'),(32,16,'Temperature'),(33,17,'Humidity'),(34,17,'Moisture'),(35,18,'Temperature'),
(36,18,'Humidity'),(37,19,'Moisture'),(38,19,'Temperature'),(39,20,'Humidity'),(40,21,'Moisture'),
(41,22,'Temperature'),(42,23,'Humidity'),(43,24,'Moisture');
GO

INSERT INTO SensorData (DataID, SensorID, Value, RecordedTime) VALUES
(1,1,15.5,'2025-04-01 10:00:00'),
(2,2,23.0,'2025-04-01 10:05:00'),
(3,3,65.0,'2025-04-01 10:10:00'),
(4,4,30.0,'2025-04-01 10:15:00'),
(5,5,21.5,'2025-04-01 10:20:00'),
(6,6,62.0,'2025-04-01 10:25:00'),
(7,7,18.5,'2025-04-01 10:30:00'),
(8,8,24.0,'2025-04-01 10:35:00'),
(9,9,57.0,'2025-04-01 10:40:00'),
(10,10,19.5,'2025-04-01 10:45:00'),
(11,11,24.0,'2025-04-01 10:50:00'),
(12,12,59.5,'2025-04-01 10:55:00'),
(13,13,17.5,'2025-04-01 11:00:00'),
(14,14,22.5,'2025-04-01 11:05:00'),
(15,15,60.5,'2025-04-01 11:10:00'),
(16,16,20.5,'2025-04-01 11:15:00'),
(17,17,22.0,'2025-04-01 11:20:00'),
(18,18,62.5,'2025-04-01 11:25:00'),
(19,19,24.0,'2025-04-01 11:30:00'),
```

```
(20,20,22.5,'2025-04-01 11:35:00'),
(21,21,61.0,'2025-04-01 11:40:00'),
(22,22,23.5,'2025-04-01 11:45:00'),
(23,23,22.0,'2025-04-01 11:50:00'),
(24,24,60.5,'2025-04-01 11:55:00'),
(25,25,23.5,'2025-04-01 12:00:00'),
(26,26,22.0,'2025-04-01 12:05:00'),
(27,27,60.0,'2025-04-01 12:10:00'),
(28,28,19.5,'2025-04-01 12:15:00'),
(29,29,24.0,'2025-04-01 12:20:00'),
(30,30,22.5,'2025-04-01 12:25:00'),
(31,31,18.0,'2025-04-01 12:30:00'),
(32,32,22.0,'2025-04-01 12:35:00'),
(33,33,62.0,'2025-04-01 12:40:00'),
(34,34,23.0,'2025-04-01 12:45:00'),
(35,35,24.0,'2025-04-01 12:50:00'),
(36,36,22.5,'2025-04-01 12:55:00'),
(37,37,23.0,'2025-04-01 13:00:00'),
(38,38,21.5,'2025-04-01 13:05:00'),
(39,39,22.5,'2025-04-01 13:10:00'),
(40,40,24.0,'2025-04-01 13:15:00'),
(41,41,22.0,'2025-04-01 13:20:00'),
(42,42,23.5,'2025-04-01 13:25:00'),
(43,43,24.0,'2025-04-01 13:30:00');
GO

INSERT INTO IrrigationSystem (IrrigationID, FieldID, Status) VALUES
(1,1,'OFF'),(2,2,'ON'),(3,3,'OFF'),(4,4,'ON'),(5,5,'OFF'),
(6,6,'ON'),(7,7,'OFF'),(8,8,'ON'),(9,9,'OFF'),(10,10,'ON'),
(11,11,'OFF'),(12,12,'ON'),(13,13,'OFF'),(14,14,'ON'),(15,15,'OFF'),
(16,16,'ON'),(17,17,'OFF'),(18,18,'ON'),(19,19,'OFF'),(20,20,'ON'),
(21,21,'OFF'),(22,22,'ON'),(23,23,'OFF'),(24,24,'ON'),
(25,25,'OFF'),(26,26,'ON'),(27,27,'OFF'),(28,28,'ON'),
(29,29,'OFF'),(30,30,'ON'),(31,31,'OFF'),(32,32,'ON'),
(33,33,'OFF'),(34,34,'ON'),(35,35,'OFF'),(36,36,'ON'),
(37,37,'OFF'),(38,38,'ON'),(39,39,'OFF'),(40,40,'ON'),
(41,41,'OFF'),(42,42,'ON'),(43,43,'OFF');
GO

INSERT INTO FieldWorker (FieldID, WorkerID) VALUES
(1,1),(1,2),(2,3),(2,4),(3,5),(3,6),(4,7),(4,8),(5,9),(5,10),
(6,11),(6,12),(7,13),(7,14),(8,15),(8,16),(9,17),(9,18),(10,19),(10,20),
(11,21),(11,22),(12,23),(12,24),(13,25),(13,26),(14,27),(14,28),(15,29),(15,30),
(16,31),(16,32),(17,33),(17,34),(18,35),(18,36),(19,37),(19,38),(20,39),(20,40),
(21,41),(21,42),(22,43);
GO

INSERT INTO Harvest (HarvestID, FieldID, CropID, Quantity, HarvestDate) VALUES
(1,1,1,200.5,'2025-07-01'),(2,2,2,150.0,'2025-07-02'),(3,3,3,180.0,'2025-07-03'),
(4,4,4,220.0,'2025-07-04'),(5,5,5,160.0,'2025-07-05'),(6,6,6,190.0,'2025-07-06'),
(7,7,7,175.0,'2025-07-07'),(8,8,8,200.0,'2025-07-08'),(9,9,9,185.0,'2025-07-09'),
(10,10,10,195.0,'2025-07-10'),(11,11,11,180.0,'2025-07-11'),(12,12,12,170.0,'2025-07-12'),
(13,13,13,160.0,'2025-07-13'),(14,14,14,155.0,'2025-07-14'),(15,15,15,165.0,'2025-07-15'),
(16,16,16,175.0,'2025-07-16'),(17,17,17,185.0,'2025-07-17'),(18,18,18,190.0,'2025-07-18'),
(19,19,19,195.0,'2025-07-19'),(20,20,20,200.0,'2025-07-20'),(21,21,21,205.0,'2025-07-21'),
(22,22,22,210.0,'2025-07-22'),(23,23,23,215.0,'2025-07-23'),(24,24,24,220.0,'2025-07-24'),
(25,25,25,225.0,'2025-07-25'),(26,26,26,230.0,'2025-07-26'),(27,27,27,235.0,'2025-07-27'),
(28,28,28,240.0,'2025-07-28'),(29,29,29,245.0,'2025-07-29'),(30,30,30,250.0,'2025-07-30'),
(31,31,31,255.0,'2025-07-31'),(32,32,32,260.0,'2025-08-01'),(33,33,33,265.0,'2025-08-02'),
```

```
(34,34,34,270.0,'2025-08-03'),(35,35,35,275.0,'2025-08-04'),(36,36,36,280.0,'2025-08-05'),
(37,37,37,285.0,'2025-08-06'),(38,38,38,290.0,'2025-08-07'),(39,39,39,295.0,'2025-08-08'),
(40,40,40,300.0,'2025-08-09'),(41,41,41,305.0,'2025-08-10'),(42,42,42,310.0,'2025-08-11'),
(43,43,43,315.0,'2025-08-12');
GO
```

DQL part

Query #	Query Name	Purpose	Description
1	View All Farms	Retrieve data	Displays all records from the Farm table
2	Crops with Growth > 100	Filtering	Shows crops that take more than 100 days to grow
3	Farms with Fields	Join	Lists farms along with their corresponding fields and area
4	Crops with Growth > 120	Filtering	Identifies crops with very long growth duration
5	High Temperature Sensor Data	Join + Filter	Retrieves temperature readings above 23
6	Total Harvest per Farm	Aggregation	Calculates total harvested quantity per farm
7	Avg Sensor Readings per Field	Aggregation	Computes average temperature, moisture, humidity per field
8	Top 5 Farms by Harvest	Sorting + Aggregation	Finds top 5 farms with highest production
9	Fields with No Irrigation ON	Conditional Aggregation	Identifies fields where irrigation is never ON
10	Crops Not Yet Harvested	Left Join	Finds planted crops that have no harvest record
11	Workers in Multiple Fields	Grouping	Shows workers assigned to more than one field
12	Max Temperature per Field	Aggregation	Finds highest temperature recorded in each field
13	Daily Avg Temperature	Aggregation + Date	Calculates average temperature per day
14	Largest Farms by Area	Aggregation + Sorting	Lists farms by total field area
15	Abnormal Sensor Conditions	Filtering	Detects unusual sensor readings (high temp / low moisture)
16	Crop Count per Farm	Aggregation	Counts distinct crops grown in each farm
17	Worker Assignments	Join	Displays which workers are assigned to which fields
18	Irrigation Status	Retrieval	Shows current ON/OFF status of irrigation systems

Table 22

```sql
-- ================================================================
-- DQL — RELATIONAL QUERIES
-- ================================================================


PRINT '================================================================';
PRINT 'DQL — RELATIONAL QUERIES';
PRINT '================================================================';
GO

-- 1. View all farms
SELECT * FROM Farm;
GO

-- 2. Crops with growth duration > 100
SELECT CropName, GrowthDuration
FROM Crop
WHERE GrowthDuration > 100;
GO

-- 3. Farms with their fields
SELECT F.FarmName, Fi.FieldID, Fi.Area
FROM Farm F
JOIN Field Fi ON F.FarmID = Fi.FarmID;
GO

-- 4. Crops with long growth duration (>120)
SELECT CropName, GrowthDuration
FROM Crop
WHERE GrowthDuration > 120;
GO

-- 5. Sensor data with high temperature
SELECT SD.SensorID, SD.Value, SD.RecordedTime
FROM SensorData SD
JOIN Sensor S ON SD.SensorID = S.SensorID
WHERE S.SensorType = 'Temperature' AND SD.Value > 23;
GO

-- 6. Total harvest per farm
SELECT F.FarmName, SUM(H.Quantity) AS TotalHarvest
FROM Harvest H
JOIN Field Fi ON H.FieldID = Fi.FieldID
JOIN Farm F ON Fi.FarmID = F.FarmID
GROUP BY F.FarmName;
GO

-- 7. Average sensor readings per field
SELECT S.FieldID,
    AVG(CASE WHEN S.SensorType = 'Temperature' THEN SD.Value END) AS AvgTemp,
    AVG(CASE WHEN S.SensorType = 'Moisture' THEN SD.Value END) AS AvgMoisture,
    AVG(CASE WHEN S.SensorType = 'Humidity' THEN SD.Value END) AS AvgHumidity
FROM SensorData SD
JOIN Sensor S ON SD.SensorID = S.SensorID
GROUP BY S.FieldID;
GO

-- 8. Top 5 farms by total harvest
SELECT TOP 5 F.FarmName, SUM(H.Quantity) AS TotalHarvest
```

```sql
FROM Harvest H
JOIN Field Fi ON H.FieldID = Fi.FieldID
JOIN Farm F ON Fi.FarmID = F.FarmID
GROUP BY F.FarmName
ORDER BY TotalHarvest DESC;
GO

-- 9. Fields with no irrigation system ON
SELECT FieldID
FROM IrrigationSystem
GROUP BY FieldID
HAVING SUM(CASE WHEN Status = 'ON' THEN 1 ELSE 0 END) = 0;
GO

-- 10. Crops planted but not yet harvested
SELECT FC.FieldID, FC.CropID
FROM FieldCrop FC
LEFT JOIN Harvest H
ON FC.FieldID = H.FieldID AND FC.CropID = H.CropID
WHERE H.HarvestID IS NULL;
GO

-- 11. Workers assigned to multiple fields
SELECT WorkerID, COUNT(FieldID) AS FieldCount
FROM FieldWorker
GROUP BY WorkerID
HAVING COUNT(FieldID) > 1;
GO



-- 12. Highest temperature recorded per field
SELECT S.FieldID, MAX(SD.Value) AS MaxTemp
FROM SensorData SD
JOIN Sensor S ON SD.SensorID = S.SensorID
WHERE S.SensorType = 'Temperature'
GROUP BY S.FieldID;
GO

-- 13. Daily average temperature
SELECT CAST(SD.RecordedTime AS DATE) AS Date,
    AVG(SD.Value) AS AvgTemp
FROM SensorData SD
JOIN Sensor S ON SD.SensorID = S.SensorID
WHERE S.SensorType = 'Temperature'
GROUP BY CAST(SD.RecordedTime AS DATE);
GO

-- 14. Farms with largest total field area
SELECT F.FarmName, SUM(Fi.Area) AS TotalArea
FROM Farm F
JOIN Field Fi ON F.FarmID = Fi.FarmID
GROUP BY F.FarmName
ORDER BY TotalArea DESC;
GO

-- 15. Sensors that recorded abnormal conditions
SELECT DISTINCT SD.SensorID
FROM SensorData SD
```

```
JOIN Sensor S ON SD.SensorID = S.SensorID
WHERE
   (S.SensorType = 'Temperature' AND SD.Value > 30)
   OR
   (S.SensorType = 'Moisture' AND SD.Value < 15);
GO

-- 16. Crop count per farm
SELECT F.FarmName, COUNT(DISTINCT FC.CropID) AS CropCount
FROM FieldCrop FC
JOIN Field Fi ON FC.FieldID = Fi.FieldID
JOIN Farm F ON Fi.FarmID = F.FarmID
GROUP BY F.FarmName;
GO

-- 17. Worker assignment details
SELECT W.Name, FW.FieldID
FROM Worker W
JOIN FieldWorker FW ON W.WorkerID = FW.WorkerID;
GO

--18. Irrigation current status
SELECT FieldID, Status
FROM IrrigationSystem;
GO
```

DCL part

DCL Overview

Role	Login Name	User Name	Purpose	Access Level
Admin	FarmAdmin	AdminUser	System administrator	Full control over database
Worker	FarmWorker	WorkerUser	Farm staff	Limited access (data operations)
Viewer	FarmViewer	ViewerUser	External viewer / analyst	Read-only access

Table 23

DCL Operations Summary

Step	Operation	Description
1	Drop Existing Logins	Removes old logins if they already exist to avoid duplication
2	Create Logins	Creates server-level authentication (FarmAdmin, FarmWorker, FarmViewer)
3	Drop Existing Users	Removes existing database users if present
4	Create Users	Maps logins to database users inside *SmartAgricultureDB*

Table 24

```sql
-- ================================================================
-- DCL — ACCESS CONTROL
-- ================================================================

PRINT '================================================================';
PRINT 'DCL — ACCESS CONTROL';
PRINT '================================================================';
GO
-- Drop logins if exist
IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'FarmAdmin')
   DROP LOGIN FarmAdmin;
GO
IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'FarmWorker')
   DROP LOGIN FarmWorker;
GO
IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'FarmViewer')
   DROP LOGIN FarmViewer;
GO

-- Create Logins
CREATE LOGIN FarmAdmin WITH PASSWORD = 'Admin@123';
CREATE LOGIN FarmWorker WITH PASSWORD = 'Worker@123';
CREATE LOGIN FarmViewer WITH PASSWORD = 'Viewer@123';
```

```
GO

USE SmartAgricultureDB;
GO

-- Drop users if exist
IF EXISTS (SELECT * FROM sys.database_principals WHERE name = 'AdminUser')
   DROP USER AdminUser;
GO
IF EXISTS (SELECT * FROM sys.database_principals WHERE name = 'WorkerUser')
   DROP USER WorkerUser;
GO
IF EXISTS (SELECT * FROM sys.database_principals WHERE name = 'ViewerUser')
   DROP USER ViewerUser;
GO

-- Create Users
CREATE USER AdminUser FOR LOGIN FarmAdmin;
CREATE USER WorkerUser FOR LOGIN FarmWorker;
CREATE USER ViewerUser FOR LOGIN FarmViewer;
GO
```

3.2 Documentations

Views

View Name	Purpose	Description
FieldSensorSummary	Aggregation	Calculates average temperature, moisture, and humidity per field
FarmTotalHarvest	Aggregation	Computes total harvest quantity for each farm
IrrigationStatus	Monitoring	Displays irrigation status and last update for each field
WorkerAssignmentSummary	Summary	Shows number of fields per worker and assigned field IDs
CriticalFieldConditions	Alerting	Identifies abnormal sensor readings (high temp, low moisture)

Table 25

Definition:

Views are virtual tables that simplify complex queries and improve readability. They allow users to access aggregated and filtered data without writing repeated SQL logic. These views support monitoring, reporting, and decision-making in the system.

Explanation:

- **FieldSensorSummary**
  This view calculates the average temperature, moisture, and humidity for each field using sensor data. It helps monitor overall environmental conditions in a simplified format.

- **FarmTotalHarvest**

  This view sums up all harvest quantities for each farm. It is used to quickly evaluate farm productivity.
- **IrrigationStatus**

  This view shows each field's area along with its irrigation status and last update time. It helps track irrigation activity across fields.
- **WorkerAssignmentSummary**

  This view counts how many fields each worker is assigned to and lists those field IDs. It provides a clear overview of worker workload.
- **CriticalFieldConditions**

  This view displays sensor readings where conditions are abnormal (high temperature or low moisture). It is used for early detection of potential issues.

Indexes

Index Name	Table	Column(s)	Purpose
IDX_SensorData_SensorID	SensorData	SensorID	Speeds up joins with Sensor
IDX_SensorData_RecordedTime	SensorData	RecordedTime	Optimizes time-based queries
IDX_Harvest_FieldID	Harvest	FieldID	Improves aggregation per field
IDX_Harvest_CropID	Harvest	CropID	Speeds crop-related queries
IDX_FieldWorker_WorkerID	FieldWorker	WorkerID	Faster worker-field lookups
IDX_Irrigation_FieldID	IrrigationSystem	FieldID	Optimizes field-based queries
IDX_Irrigation_Status	IrrigationSystem	Status	Speeds status filtering
IDX_FieldCrop_FieldID_CropID	FieldCrop	(FieldID, CropID)	Optimizes join for harvest checks

Table 26

Definition:

Indexes improve database performance by reducing the time required to retrieve data. They are especially useful for large datasets and frequently queried columns. In this system, indexes optimize joins, filtering, and aggregation operations.

Explanation:

- **IDX_SensorData_SensorID**
  Speeds up queries that join SensorData with Sensor using SensorID.
- **IDX_SensorData_RecordedTime**
  Improves performance of queries filtering or sorting by time.

- **IDX_Harvest_FieldID**

  Helps faster calculation of total harvest per field.
- **IDX_Harvest_CropID**

  Optimizes queries related to specific crops.
- **IDX_FieldWorker_WorkerID**

  Speeds up retrieval of fields assigned to a specific worker.
- **IDX_Irrigation_FieldID**

  Improves lookup of irrigation data per field.
- **IDX_Irrigation_Status**

  Makes filtering fields by ON/OFF irrigation faster.
- **IDX_FieldCrop_FieldID_CropID**

  Optimizes queries checking planted vs harvested crops.

Triggers

Trigger Name	Type	Table	Purpose
TR_UpdateIrrigationLastUpdated	AFTER UPDATE	IrrigationSystem	Updates timestamp when irrigation status changes
TR_ValidateHarvestQuantity	INSTEAD OF INSERT	Harvest	Prevents negative harvest quantities
TR_AutoIrrigation	AFTER INSERT	SensorData	Automatically turns ON irrigation if moisture is low
TR_LogSensorAnomalies	AFTER INSERT	SensorData	Logs abnormal sensor readings into SensorAlerts
TR_WorkerFieldLimit	INSTEAD OF INSERT	FieldWorker	Limits workers to max 3 fields

Table 27

Definition:

Triggers automatically execute in response to specific database events such as INSERT or UPDATE. They enforce business rules and maintain data integrity without manual intervention. In this project, triggers handle automation (irrigation), validation (harvest), and monitoring (alerts).

Explanation:

- **TR_UpdateIrrigationLastUpdated**
  Automatically updates the LastUpdated timestamp whenever irrigation status changes.
- **TR_ValidateHarvestQuantity**
  Prevents inserting harvest records with negative quantities to ensure valid data.
- **TR_AutoIrrigation**
  Automatically turns irrigation ON if a newly inserted moisture reading is below 20.
- **TR_LogSensorAnomalies**
  Inserts abnormal sensor readings into the SensorAlerts table for monitoring.
- **TR_WorkerFieldLimit**
  Prevents assigning a worker to more than 3 fields, enforcing workload limits.

Stored Procedures

Procedure Name	Purpose	Description
GetFarmHarvest	Reporting	Returns total harvest per farm
GetFieldSensorData	Monitoring	Retrieves sensor readings for a specific field
GetWorkerFields	Management	Shows fields assigned to a worker
GetIrrigationStatus	Monitoring	Displays irrigation status for all fields
AddHarvest	Insert	Adds new harvest record
AddSensorData	Insert	Inserts new sensor reading
UpdateIrrigationStatus	Control	Updates irrigation based on moisture condition
GetUnharvestedCrops	Analysis	Finds crops not yet harvested
GetTopFarms	Ranking	Returns top N farms by harvest

Table 28

Definition:

Stored procedures encapsulate reusable SQL logic and improve efficiency and security. They allow parameterized operations and reduce redundancy in queries. In this system, they support reporting, data insertion, and automated decision-making.

Explanation:

- **GetFarmHarvest**
  Returns total harvest quantity for each farm.
- **GetFieldSensorData (@FieldID)**
  Retrieves all sensor readings for a specific field ordered by time.
- **GetWorkerFields (@WorkerID)**
  Shows all fields assigned to a specific worker along with farm details.
- **GetIrrigationStatus**
  Displays irrigation status and last update time for all fields.

- **AddHarvest**

  Inserts a new harvest record into the database.

- **AddSensorData**

  Adds new sensor readings to the system.

- **UpdateIrrigationStatus (@FieldID, @NewStatus)**

  Updates irrigation status but prevents turning it ON if moisture is already high.

- **GetUnharvestedCrops**

  Lists crops that are planted but not yet harvested.

- **GetTopFarms (@TopN)**

  Returns top N farms based on total harvest quantity.

3.3 Database deployment description

```sql
PRINT '==============================================================';
PRINT 'DATABASE DEPLOYMENT SCRIPT';
PRINT '==============================================================';
GO

USE master;
GO

IF DB_ID('SmartAgricultureDB') IS NOT NULL
BEGIN
    ALTER DATABASE SmartAgricultureDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE SmartAgricultureDB;
END
GO

CREATE DATABASE SmartAgricultureDB;
GO

USE SmartAgricultureDB;
GO
```

Figure 9

Purpose:

This deployment script ensures a clean and consistent database setup by removing any existing version and recreating it from scratch. It is useful for testing, development, and resetting the system to a known state. This guarantees that all subsequent scripts run without conflicts or leftover data.

Explanation:

- **USE master**
  The script starts by switching to the *master* database to safely perform administrative operations.
- **IF DB_ID('SmartAgricultureDB') IS NOT NULL**
  Checks whether the database already exists in the system.
- **SET SINGLE_USER WITH ROLLBACK IMMEDIATE**
  Forces all active connections to close and rolls back any ongoing transactions to avoid conflicts.

- **DROP DATABASE SmartAgricultureDB**

  Deletes the existing database completely.
- **CREATE DATABASE SmartAgricultureDB**

  Creates a new, clean database instance.
- **USE SmartAgricultureDB**

  Switches the working context to the newly created database for further operations (DDL, DML, etc.).

Project conclusions and results

The Smart Agriculture Database System successfully demonstrates how modern data management techniques can be applied to improve agricultural operations. The system effectively integrates multiple components such as farm management, sensor data collection, irrigation control, and harvest tracking into a unified and structured database. Through the use of relational tables, constraints, and stored procedures, data integrity and consistency are ensured across all operations.

The implementation of automated processes, such as sensor data recording and irrigation control based on real-time conditions, highlights the system's ability to support intelligent decision-making. Additionally, the use of views and queries enables efficient data analysis, allowing users to monitor field conditions, evaluate crop performance, and generate meaningful reports.

Overall, the project achieves its objective of designing a reliable and scalable database system for smart agriculture. It provides a strong foundation for future enhancements, such as real-time monitoring applications, predictive analytics, and integration with IoT devices, making it a practical solution for modern agricultural management.